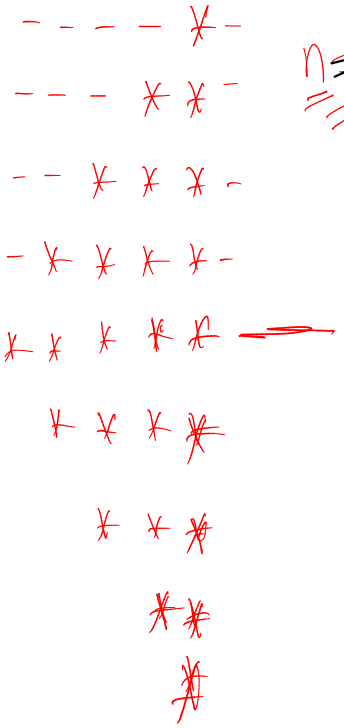
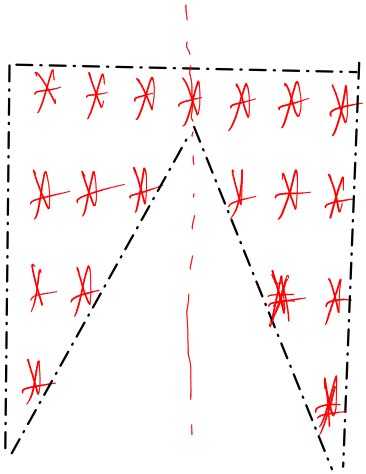


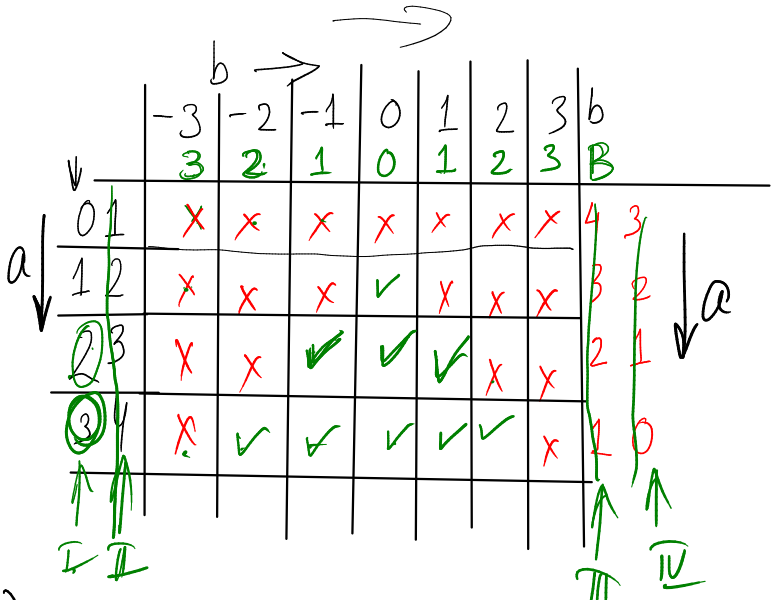
n=4

a	A	b=0	1	2	3	4
			1	2	3	4
-4	4	✓	✓	✓	✓	X
-3	3	✓	✓	✓	X	X
-2	2	✓	✓	X	X	X
-1	1	✓	X	X	X	X
0	0	X	X	X	X	X
1	1					
2	2					
3	3					
4	4					
		5	4	3	2	1

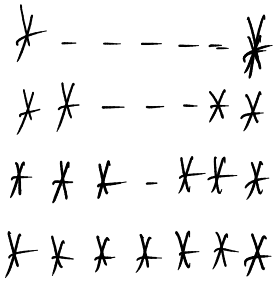
4 3 2 1 0



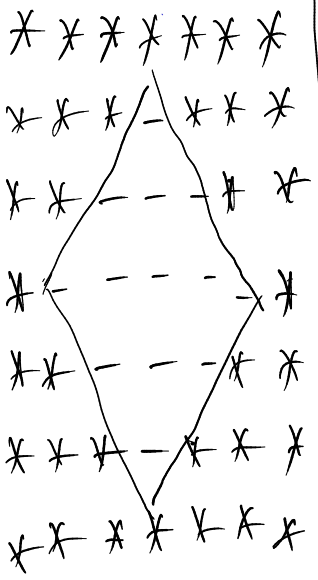
n=3



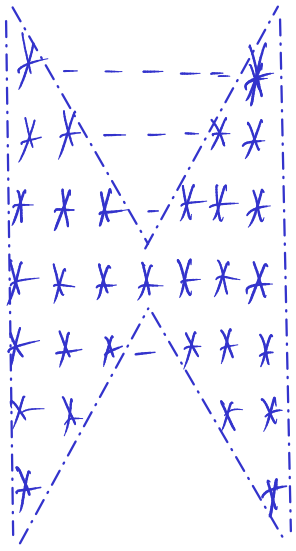

```
for(a=0; a<=n; a++){
    for(b=-n; b<=n; b++){
        B=(b<0)?-b:b;
        If(B<a) Pf("  ");
        else    Pf("%c", '*');
    }
    Pf("\n");
}
```



I



II 2 loops



III 2 loops

switch ... case

Syntax :

⇒ Branching

switch(expr){
Keyword

case 1: ≡
Key
break;

case 2: ≡
break;

⋮

default: ≡
~~break;~~

}

```
int V;
≡
sf("%d", &V);
if(V==1)
        A —
else if(V==2)
        B —
else if(V==3)
        C —
else
        D —
```

else...if ladder

→

```
switch(V)
{
case 1: A —
        break;
case 2: B —
        break;
case 3: C —
        break;
default : D —
}
```

```
switch(V) {
case 1: V+=10;
case 2: V+=20;
case 3: V+=30;
default: V+=100;
}
pf("%d", V);
```

→ the case must ~~NOT~~ contain Integral constant only.

```
case 'A':
case 'B':
case '*':
```

```
case -3:
case 0:
case 'a':
case -20:
```

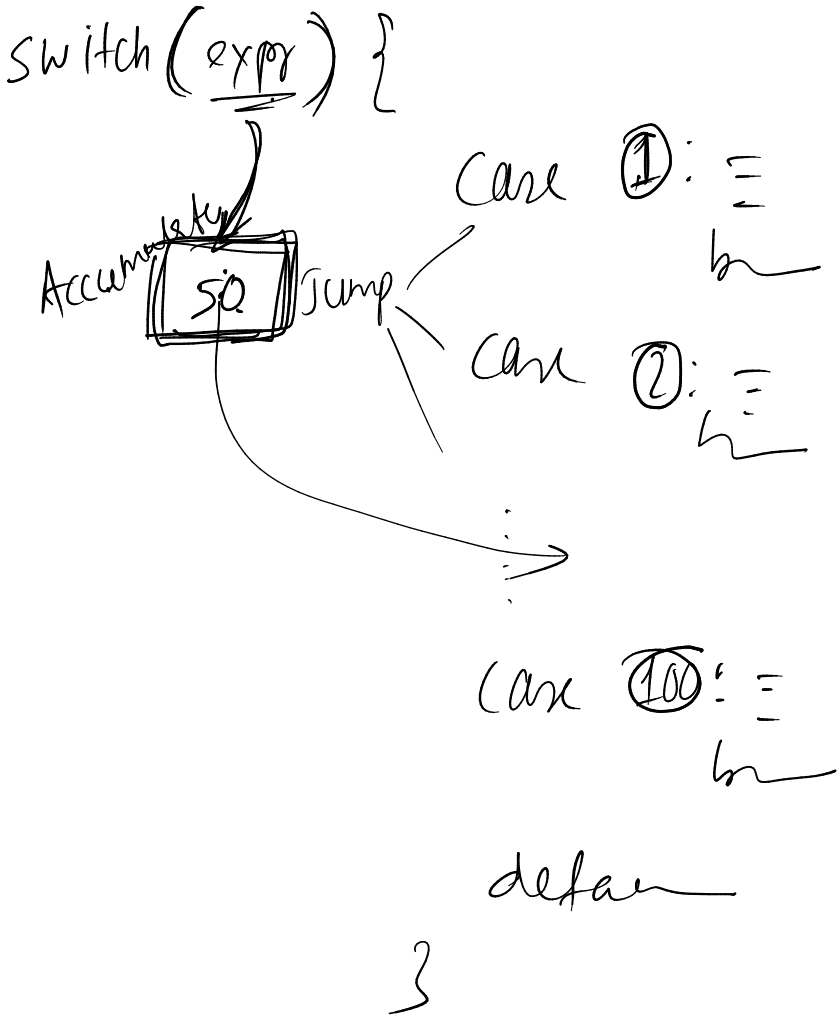
```
case 0:
case 30.0:
case 40.0:
```

58

```
if(Perc >=75)
    Pf("Distinction\n");
else if(Perc >=60)
    1st
else if(Perc >=50)
    2nd
else if (Perc >=40)
    3rd
else
    Pf("Fail");
```

⇒ //

```
switch((Perc))
{
case 100:
case 99:
case 98:
.
.
.
75: ____;
    ____
case 74:
}
```


switch (expr) {
Accumulator
50
Jump
case 1: =
b
case 2: =
b
case 100: =
b
defa
}